
535510 RL Team Project Baseline Evaluation and Toy Tasks:

Selective and Cost-Aware RL Query Rewriting for Retrieval-Augmented Generation

蔡尚融 李昇昕 李政緯
Department of Computer Science
National Yang Ming Chiao Tung University
shangjung.cs12@nycu.edu.tw
hsin.cs12@nycu.edu.tw
cody.weilee@gmail.com

1 Project Overview

1.1 Profile

Track: 1. Algorithm

Abstract and project goal: Retrieval-augmented generation (RAG) improves question answering by retrieving external evidence before answer generation, but the overall performance is highly sensitive to the input query. Existing query rewriting methods have shown that reformulating user questions can improve both retrieval and downstream answering, yet many approaches rewrite every query indiscriminately or optimize only a single stage of the pipeline. This is suboptimal because some queries are already retrieval-friendly, while others need targeted operations such as entity expansion, ambiguity resolution, temporal clarification, or decomposition into sub-queries. In this project, we propose a selective and cost-aware reinforcement learning framework for query rewriting in RAG. Instead of always rewriting the input, our method learns a lightweight policy that decides whether to keep the original query or apply one of several rewrite operations. The reward integrates retrieval quality and answer quality while penalizing excessive rewrite length and retrieval cost. We plan to evaluate the method on both single-hop and multi-hop question answering benchmarks and focus on the effectiveness-efficiency trade-off under limited compute.

TL;DR (“Too Long; Didn’t Read”): A lightweight RL-based RAG system that learns when and how to rewrite a query, improving answer quality while controlling rewrite and retrieval cost.

1.2 Motivation

We are interested in this topic because modern RAG systems often fail not because the reader is weak, but because the input query is ambiguous, underspecified, conversational, or poorly aligned with the retriever’s indexing space. Prior work shows that moving from retrieve-then-read to rewrite-retrieve-read can substantially improve knowledge-intensive QA [Ma et al., 2023], and recent work such as RQ-RAG further demonstrates the benefits of refinement, decomposition, and disambiguation for both single-hop and multi-hop QA [Chan et al., 2024]. However, rewriting every query is not always beneficial and may increase token and retrieval costs without improving the final answer.

- Why is the problem interesting? The project studies a practical bottleneck in RAG: whether the system should rewrite a query at all, and if so, what kind of rewrite is most useful for

retrieval and answer generation. This makes the problem both scientifically meaningful and practically relevant for efficient QA systems.

- **Critical challenges.** Challenge 1: Not every query benefits from rewriting, and unnecessary rewriting may distort the user’s intent or introduce retrieval drift. Challenge 2: Query rewriting should optimize the full RAG pipeline, not retrieval alone, because better retrieval scores do not always translate into better final answers. Challenge 3: Sequence-level RL for free-form text generation can be expensive, so the method must remain lightweight enough to run on limited hardware.
- **Why does the problem remain open or unsolved?** Seminal RAG and dense retrieval work established the retrieve-then-read paradigm [Lewis et al., 2020, Karpukhin et al., 2020]. More recent work showed that query rewriting is powerful in retrieval-augmented LLM pipelines [Ma et al., 2023], that query refinement and decomposition can further improve RAG [Chan et al., 2024], and that multi-aspect feedback can benefit RL-based rewriting [Wang et al., 2024]. Even so, there is still a practical gap between strong but heavier rewriting pipelines and lightweight methods that explicitly decide *whether* to rewrite while accounting for system cost.
- **State-of-the-art methods in our opinion.** For the specific subproblem of query refinement in RAG with public code and reproducible setup, we consider RQ-RAG [Chan et al., 2024] the strongest reference baseline. For reward design in RL-based rewriting, MaFeRw [Wang et al., 2024] is a strong recent reference.

2 Problem Formulation

We formulate query rewriting for RAG as a contextual bandit, or equivalently a one-step Markov decision process. For each input question q , the agent observes a state $s(q)$ derived from the original query and optional shallow retrieval statistics from an initial retrieval pass, such as top- k score gaps, entropy, query length, and named-entity indicators. The agent then chooses an action $a \in \mathcal{A}$ from a finite rewrite action space, for example: keep the original query, expand entities or keywords, add temporal clarification, disambiguate an ambiguous term, compress the query into keyword style, or decompose the question into two sub-queries. The chosen action is converted into a rewritten query $\tilde{q} = f(q, a)$, which is then sent to a fixed retriever and a fixed reader. This setting uses standard scalar reward feedback and keeps the environment lightweight enough for a single-GPU implementation.

Formally, let $\mathcal{D}$ denote the question distribution, $R(\tilde{q})$ denote the top- k retrieved passages for the rewritten query, and $G(q, R(\tilde{q}))$ denote the final generated answer. We learn a policy $\pi_\theta(a | s)$ that maximizes

$$\mathbb{E}_{q \sim \mathcal{D}, a \sim \pi_\theta(\cdot | s(q))} [\alpha \cdot \text{AnsScore} + \beta \cdot \text{RetScore} - \lambda_1 \cdot \text{RewriteCost} - \lambda_2 \cdot \text{ContextCost}]. \quad (1)$$

Here, AnsScore can be Exact Match (EM) or token-level F1, RetScore can be Recall@ k , MRR, or nDCG, and the cost terms penalize overly long rewrites and excessive retrieved context. We assume the retriever and reader remain fixed during the main RL training stage so that the observed gains can be attributed primarily to the rewrite policy rather than to changes in other modules.

3 Empirical Evaluation

We will evaluate the proposed method from three perspectives: retrieval quality, downstream QA quality, and efficiency.

3.1 Performance Metrics

We plan to report the following key metrics.

- **Retrieval quality:** Recall@ k , MRR, and nDCG@ k . These metrics measure whether the rewritten query helps retrieve more relevant evidence and rank it higher.
- **Downstream QA quality:** Exact Match (EM) and token-level F1. These metrics verify whether better retrieval actually improves the final answer.

- Efficiency: average rewrite length, average retrieved context length, and average retrieval calls or latency proxy. These metrics are important because our goal is not only to improve answer quality, but also to control rewrite and retrieval cost.

The efficiency metrics are necessary because a method that always rewrites aggressively may achieve slightly higher accuracy at the expense of much higher token and retrieval overhead.

3.2 Baseline Methods

We will compare against the following baselines.

- Vanilla RAG [Lewis et al., 2020]: standard retrieve-then-read without query rewriting.
- BM25 and DPR-based RAG pipelines [Karpukhin et al., 2020]: standard sparse and dense retrieval variants used as fixed retrieval backbones for the same downstream reader.
- Rewrite-Retrieve-Read [Ma et al., 2023]: a trainable query rewriting framework that replaces retrieve-then-read with rewrite-retrieve-read.
- RQ-RAG [Chan et al., 2024]: a recent refinement-based RAG method with explicit rewriting, decomposition, and disambiguation.
- Prompt-based rewriting: a non-RL baseline that uses manually designed prompts or heuristics to rewrite every query.
- MaFeRw-inspired reward-guided rewriting [Wang et al., 2024]: if compute permits, we will include a simplified RL baseline or ablation inspired by multi-aspect reward feedback.

3.3 Benchmark Tasks or Datasets

We plan to evaluate on one single-hop QA dataset and one multi-hop QA dataset.

- Natural Questions (NQ) [Kwiatkowski et al., 2019]: a standard open-domain single-hop QA benchmark widely used in retrieval-augmented QA.
- HotpotQA [Yang et al., 2018]: a multi-hop QA benchmark with supporting-fact supervision, suitable for evaluating decomposition and evidence selection.

If time permits, we may additionally test on 2WikiMultihopQA as an extra multi-hop benchmark. For feasibility, we will begin with smaller train and validation subsets, then scale up after the retrieval and RL pipelines become stable.

4 Methodology (Optional)

Our method contains four components. First, we use a fixed retriever-reader backbone, such as BM25 or a small dense retriever together with a fixed reader, so that the main experimental variable is the rewrite policy itself. Second, instead of performing unrestricted free-form generation, we define a small action space for selective rewriting: keep the original query, expand entities or keywords, add temporal clarification, disambiguate a term, compress the question into keyword style, or decompose the question into two sub-queries. Third, we design a cost-aware reward that combines answer quality and retrieval quality with penalties on rewrite length and retrieved context length, encouraging the policy to rewrite only when the gain justifies the cost. Finally, for feasibility, we start from a lightweight contextual-bandit or simplified policy-gradient formulation rather than sequence-level PPO on a large language model. This design keeps the project analyzable, reproducible, and suitable for limited hardware while preserving the core RL decision problem.

5 Experimental Results

5.1 Midterm Goal and Toy Environment

For the midterm milestone, we focus on the course requirement of running baselines and a small-scale task that exposes the essence of the proposed problem. We build a reproducible toy retrieval

Table 1: HotpotQA toy retrieval results on 120 held-out examples. Train-best fixed rewrite is selected only from the training split, while the oracle row uses per-example ground-truth feedback and serves as an upper bound.

Method	Recall@5	MRR	nDCG@5	Reward	Cost	Calls
Vanilla BM25	0.804	0.851	0.742	1.229	0.000	1.000
Rewrite-all keyword	0.829	0.896	0.780	1.277	0.000	1.000
Rewrite-all entity expansion	0.825	0.891	0.778	1.220	0.051	1.000
Train-best fixed rewrite	0.829	0.896	0.780	1.277	0.000	1.000
Selective bandit	0.821	0.883	0.770	1.262	0.000	1.000
Oracle best action	0.875	0.926	0.815	1.332	0.006	1.042

environment from the HotpotQA distractor development set [Yang et al., 2018]. Each question is paired with ten candidate Wikipedia paragraphs from the original dataset. We treat each paragraph title plus its sentences as a retrievable passage, and we use the supporting-fact titles as gold relevant documents. This toy setting is intentionally smaller than the final RAG pipeline, but it preserves the key difficulty of our proposal: multi-hop questions often require selecting the right query form before retrieval can recover all required evidence.

We sample 300 examples with random seed 42, use 180 examples for training the lightweight contextual-bandit selector, and evaluate on 120 held-out examples. All methods use the same BM25 retriever and the same candidate passage pool per question. Since this midterm experiment is designed to test retrieval behavior before adding a reader, we report Recall@5, MRR, nDCG@5, retrieval calls, rewrite cost, and the cost-aware reward

$$r = \text{Recall@5} + 0.5 \cdot \text{MRR} - \text{RewriteCost}.$$

The rewrite cost penalizes extra rewrite tokens and additional retrieval calls. This reward is a toy version of the answer-plus-retrieval-plus-cost objective described in Section 4. For the one-step contextual-bandit experiment, retrieval calls count the final retrieval issued by the selected action. For the multi-step extension, retrieval calls include the initial observation retrieval plus any refinement retrievals, because the policy explicitly observes retrieval feedback before deciding whether to continue.

5.2 Baselines and One-Step Toy Method

We compare six systems. **Vanilla BM25** retrieves with the original question and serves as the no-rewrite baseline. **Rewrite-all keyword** removes common stop words and retrieves with a compressed keyword-style query. **Rewrite-all entity expansion** extracts capitalized spans from the question and appends content words. These two rewrite-all baselines test whether simply rewriting every query is enough. **Train-best fixed rewrite** is a stronger non-oracle baseline: it chooses the single rewrite action with the highest average reward on the training split and then applies that fixed action to every held-out question. **Oracle best action** chooses the best action per example using ground-truth feedback and is not a deployable method; it estimates how much headroom exists in the toy action space. **Selective bandit** is our proposed toy method. It observes query length, wh-word type, number of capitalized spans, and shallow initial BM25 score statistics, then selects one action from {keep, keyword_compress, entity_expand, decompose}.

5.3 Multi-Step RL Extension

To make the midterm experiment closer to a sequential RL problem, we also implement a two-step finite-horizon MDP extension. The agent first retrieves with the current query, observes retrieval feedback, and then chooses either stop or one rewrite action. If it does not stop, it retrieves again and can make one additional decision. In addition to the one-step rewrite actions, this extension includes two feedback-aware actions: `feedback_expand`, which appends the top retrieved titles to the query, and `title_bridge`, which forms bridge queries from the top retrieved titles. We train this policy with fitted Q iteration using a k-nearest-neighbor Q estimator. We also include **Train-best fixed trace**, which selects the best fixed two-step action trace on the training split and applies that same trace to every test question. This is still a lightweight toy method, but unlike the fixed-trace baseline, the contextual bandit and FQI policies make per-question decisions from state features.

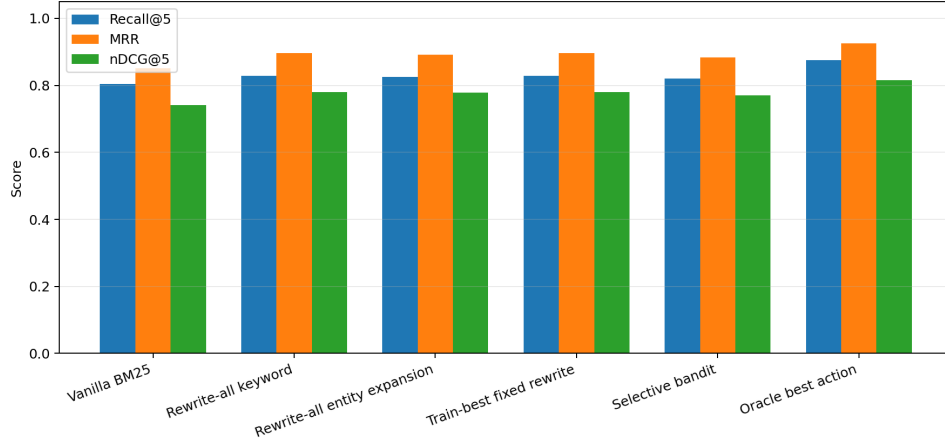


Figure 1: Retrieval metrics for baselines and the proposed toy selector. The train-best fixed baseline selects keyword compression, but the oracle gap shows that no single rewrite action is uniformly optimal.

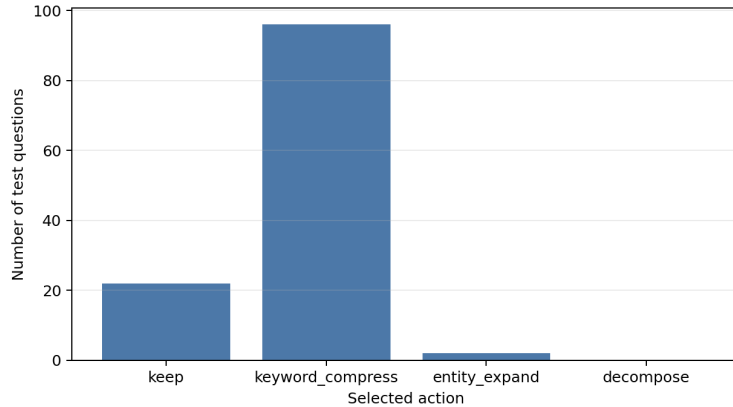


Figure 2: Actions selected by the contextual bandit on the held-out set. The policy mostly chooses keyword compression but keeps the original query for a nontrivial subset, supporting the selective-rewriting motivation.

5.4 Discussion

The results in Table 1 show why the baseline is insufficient. Vanilla BM25 retrieves only 80.4% of the required supporting documents at top 5 on this controlled toy task. This is already before answer generation, so any missing evidence directly limits the downstream reader. Train-best fixed rewrite selects `keyword_compress` on the training split and improves Recall@5 to 82.9%, confirming that query reformulation helps even when the fixed action is selected without test feedback. However, the oracle best action reaches 87.5% Recall@5 and a higher reward, which means a fixed rewrite action still leaves substantial evidence-recovery headroom.

The proposed selective bandit reaches 82.1% Recall@5 and 77.0% nDCG@5, improving over vanilla BM25 while keeping the average retrieval calls at 1.0 and the rewrite cost near zero. It does not yet surpass the strongest rewrite-all keyword baseline in raw retrieval metrics, which is useful diagnostic feedback for the next stage: our current features can identify many cases where keyword compression is useful, but they are not yet expressive enough to consistently choose entity expansion or decomposition when those actions are better. This midterm result therefore supports the central claim of the project while also identifying the main remaining work: better state features and reward learning are needed to close the gap between the learned selector and the oracle action selector.

Table 2: Two-step HotpotQA toy results. Train-best fixed trace is selected on the training split. Multi-step FQI outperforms the fixed baselines in Recall@5, showing that retrieval feedback across steps can improve evidence recovery.

Method	Recall@5	MRR	nDCG@5	Reward	Cost	Calls
Vanilla BM25	0.804	0.851	0.742	1.229	0.000	1.000
Rewrite-all keyword	0.829	0.896	0.780	1.257	0.020	2.000
Rewrite-all entity	0.825	0.891	0.778	1.238	0.033	2.000
Train-best fixed trace	0.829	0.896	0.780	1.257	0.020	2.000
Multi-step FQI	0.838	0.882	0.780	1.242	0.036	2.358
Oracle two-step	0.933	0.946	0.857	1.389	0.018	1.667

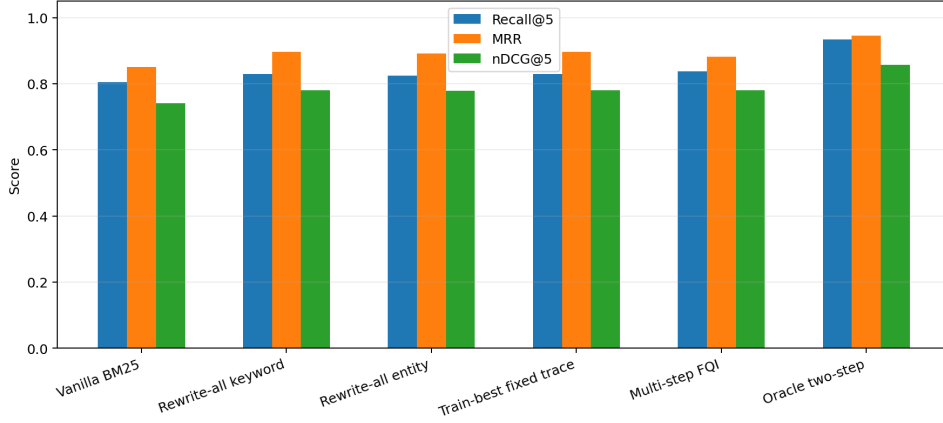


Figure 3: Retrieval metrics for the multi-step extension. The learned two-step FQI policy improves Recall@5 over vanilla BM25 and rewrite-all keyword retrieval, while the oracle two-step result indicates substantial remaining room for better policy learning.

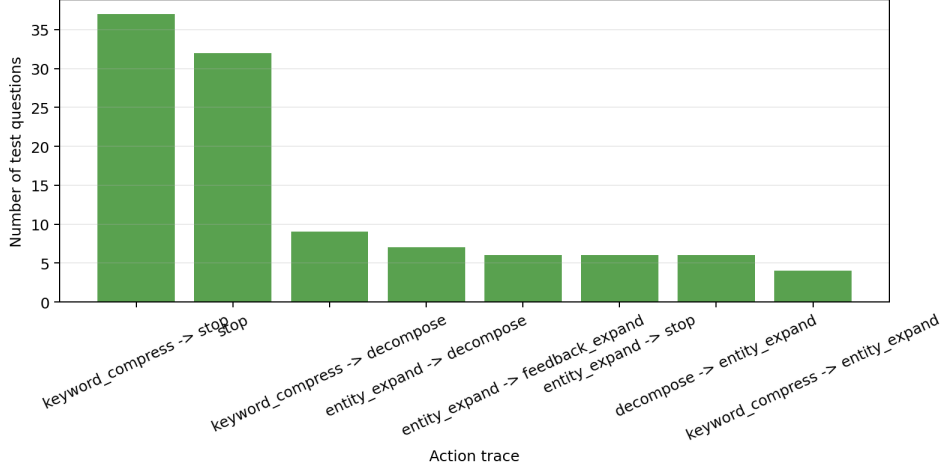


Figure 4: Most frequent action traces selected by the multi-step policy. The policy learns a mixture of stopping, one-step keyword compression, and second-step refinement actions, which is the sequential behavior intended by the proposed RL formulation.

The multi-step extension strengthens the result. As shown in Table 2, the train-best fixed trace is keyword_compress -> stop, which matches the strongest one-step fixed rewrite under the multi-step cost accounting. The two-step FQI policy reaches 83.8% Recall@5, exceeding vanilla

BM25 and the train-best fixed trace in evidence recovery. This improvement comes from using retrieval feedback as part of the next state, rather than committing to a single rewrite before seeing any evidence. The trade-off is cost: the policy uses 2.36 retrieval calls on average, compared with 2.0 calls for the fixed trace and 1.0 call for vanilla BM25. This is consistent with our cost-aware framing: sequential refinement can recover more evidence, but the final system must learn when the extra retrieval call is worth it. The two-step oracle reaches 93.3% Recall@5, indicating that the action space has significant headroom and that stronger state representations should be valuable in the final project.

A qualitative example illustrates the failure mode. For the question “Which author lived longer, Nelson Algren or Nathanael West?”, vanilla BM25 retrieves Nathanael West but misses Nelson Algren in the top 5. Keyword compression retrieves both supporting titles, improving Recall@5 from 0.5 to 1.0. This behavior matches our motivation: multi-hop comparison questions often need a query form that emphasizes both entities instead of the original conversational phrasing.

6 Expected Contributions of Each Team Member

We expect to collaborate closely on experiment design, paper writing, and weekly integration of code and results. The tentative division of responsibilities is as follows.

- 蔡尚融: lead the retriever-reader baseline pipeline, dataset preprocessing, and evaluation scripts.
- 李羿昕: implement the RL rewrite policy, state features, reward design, and training framework.
- 李政緯: reproduce external baselines such as RQ-RAG or prompt rewriting, conduct ablations, analyze the results, and help prepare the final report and presentation.

We understand that the exact division of work may change after the project starts, so we will revisit the task allocation as the experiments progress and re-balance the workload when necessary.

References

- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.322. URL <https://aclanthology.org/2023.emnlp-main.322/>.
- Charmaine Ming Kai Chan, Ricky Lo, Kai Yiu Chan, Haze Cheng, Weizhong Yao, Yuxi Zhang, Shelby Heinecke, Joshua Goh, Weiping Si, Jianguo Yu, et al. Rq-rag: Learning to refine queries for retrieval augmented generation. *arXiv preprint arXiv:2404.00610*, 2024. URL <https://arxiv.org/abs/2404.00610>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL <https://aclanthology.org/2020.emnlp-main.550/>.
- Yujing Wang, Hainan Zhang, Liang Pang, Binghui Guo, Hongwei Zheng, and Zhiming Zheng. Maferw: Query rewriting with multi-aspect feedbacks for retrieval-augmented large language models. *arXiv preprint arXiv:2408.17072*, 2024. URL <https://arxiv.org/abs/2408.17072>.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tac1_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259/>.